

GRAPH THEORY ESSENTIALS

Fundamentals of Non-Linear Linked Structures

1. Introduction to Graphs

A Graph is a powerful non-linear data structure consisting of **Vertices** (nodes) and **Edges** (connections). Unlike trees, graphs do not have a strict parent-child hierarchy, allowing for complex, multi-directional relationships.

Real-World Applications: Social networks (friendships), navigation (Google Maps), and web indexing (PageRank algorithm).

2. Essential Graph Properties

Graphs are categorized by their behavior and structure:

- **Directed vs. Undirected:** In Directed graphs (Digraphs), edges have a specific 'from-to' flow. Undirected edges are bidirectional.
- **Weighted vs. Unweighted:** Weighted graphs assign values to edges (e.g., distance, cost, or time).
- **Connected:** A graph is connected if there is a path between every pair of vertices.
- **Cyclic vs. Acyclic:** A cyclic graph contains at least one path that starts and ends at the same vertex.

3. Adjacency Matrix Representation

An Adjacency Matrix is a 2D array of size $V \times V$ (where V is the number of vertices). A value at `matrix[i][j]` indicates an edge between vertex i and j .

Pros: Fast edge lookup ($O(1)$).

Cons: High space complexity ($O(V^2)$), inefficient for 'sparse' graphs.

For weighted graphs, the matrix stores the weight value instead of 1; for unweighted graphs, it stores 0 or 1.

4. Adjacency List Representation

An Adjacency List uses an array of linked lists (or dynamic arrays). Each index represents a vertex, and its value is a list of all its neighboring nodes.

```
# Example Adjacency List (Python Dictionary)
graph = {
    'A': ['B', 'C', 'D'],
    'B': ['A', 'C'],
    'C': ['A', 'B'],
    'D': ['A']
}
```

This is highly space-efficient for **sparse graphs** (where edges are much fewer than V^2).

5. Comparing Representations

Feature	Adjacency Matrix	Adjacency List
Space Complexity	$O(V^2)$	$O(V + E)$

Edge Lookup	$O(1)$	$O(V)$ worst case
Adding Vertex	$O(V^2)$	$O(1)$
Adding Edge	$O(1)$	$O(1)$

CodeHive

Graph Theory: Deep Concepts (Part 1)

Understanding Graph density is key to choosing a representation. A graph is considered 'dense' if the number of edges E is close to the maximum possible number of edges $V(V-1)/2$.

Design Choice: In most modern high-scale applications (like Facebook's social graph), **Adjacency Lists** are preferred because most people are not friends with everyone else, making the matrix extremely wasteful.

Special Graph Types

DAG (Directed Acyclic Graph): A crucial structure in computer science used for representing task dependencies, build systems (like Make or Gradle), and data processing pipelines.

Bipartite Graphs: A set of vertices decomposed into two disjoint sets such that no two graph vertices within the same set are adjacent.

Graph Theory: Deep Concepts (Part 2)

Understanding Graph density is key to choosing a representation. A graph is considered 'dense' if the number of edges E is close to the maximum possible number of edges $V(V-1)/2$.

Design Choice: In most modern high-scale applications (like Facebook's social graph), **Adjacency Lists** are preferred because most people are not friends with everyone else, making the matrix extremely wasteful.

Special Graph Types

DAG (Directed Acyclic Graph): A crucial structure in computer science used for representing task dependencies, build systems (like Make or Gradle), and data processing pipelines.

Bipartite Graphs: A set of vertices decomposed into two disjoint sets such that no two graph vertices within the same set are adjacent.

Graph Theory: Deep Concepts (Part 3)

Understanding Graph density is key to choosing a representation. A graph is considered 'dense' if the number of edges E is close to the maximum possible number of edges $V(V-1)/2$.

Design Choice: In most modern high-scale applications (like Facebook's social graph), **Adjacency Lists** are preferred because most people are not friends with everyone else, making the matrix extremely wasteful.

Special Graph Types

DAG (Directed Acyclic Graph): A crucial structure in computer science used for representing task dependencies, build systems (like Make or Gradle), and data processing pipelines.

Bipartite Graphs: A set of vertices decomposed into two disjoint sets such that no two graph vertices within the same set are adjacent.

Graph Theory: Deep Concepts (Part 4)

Understanding Graph density is key to choosing a representation. A graph is considered 'dense' if the number of edges E is close to the maximum possible number of edges $V(V-1)/2$.

Design Choice: In most modern high-scale applications (like Facebook's social graph), **Adjacency Lists** are preferred because most people are not friends with everyone else, making the matrix extremely wasteful.

Special Graph Types

DAG (Directed Acyclic Graph): A crucial structure in computer science used for representing task dependencies, build systems (like Make or Gradle), and data processing pipelines.

Bipartite Graphs: A set of vertices decomposed into two disjoint sets such that no two graph vertices within the same set are adjacent.